FoodFlow Flask Backend Guide

How the frontend, Flask backend, and MySQL database work together

HTML, CSS and JavaScript	Flask API and sessions	Existing MySQL database

Technology used: Python Flask with mysql-connector-python. The project does not use Django and it does not recreate the existing database.

Prepared for the Online Food Ordering and Delivery Database System

1. Overall Architecture

FoodFlow follows a three-layer web application design. The browser displays the interface, JavaScript calls the Flask API, and Flask uses parameterized MySQL queries to read or write the existing schema. Flask sends JSON responses back to the browser, which updates the page without reloading it.

Browser UI	fetch() request	Flask route	MySQL query	JSON response	Updated UI

Layer	Files and responsibility
Frontend	**templates/index.html** builds the screen. **static/css/style.css** controls the visual design. **static/js/app.js** sends requests and renders returned data.
Backend	**app.py** contains Flask routes, database connection code, validation, error handling, checkout logic, and login sessions. **admin_routes.py** groups administrator operations in a Flask Blueprint.
Database	MySQL stores CUSTOMER, RESTAURANT, MENU_ITEM, ORDERS, ORDER_ITEM, PAYMENT, and DELIVERY. Views, triggers, constraints, and stored procedures remain in MySQL.

2. Flask Application Startup

When you run **python app.py**, Python starts the Flask development server at **http://127.0.0.1:5000**. Opening this address calls the **/** route. Flask renders index.html and the browser then loads the CSS and JavaScript files.

Configuration and connection

The application uses **python-dotenv** to load values from the local .env file. This keeps the MySQL password and administrator login outside the source code. db_config() reads MYSQL_HOST, MYSQL_PORT, MYSQL_DATABASE, MYSQL_USER, and MYSQL_PASSWORD.

connection() creates mysql.connector.connect(**db_config()). It is a context manager: it opens a connection for one request, then always closes it after that request finishes. This avoids keeping unused connections open.

Example configuration - do not put real passwords into .env.example

```
MYSQL_DATABASE=food_ordering_db
MYSQL_USER=root
MYSQL_PASSWORD=your_existing_mysql_password
APP_ADMIN_USERNAME=admin
APP_ADMIN_PASSWORD=your_private_password
```

3. Frontend to Flask: API Requests

The JavaScript file defines a helper named api(path, options). It uses the browser fetch() function to send an HTTP request to a Flask route. The helper sets Content-Type: application/json, reads the JSON response, and throws a readable error when the response is not successful.

User action	JavaScript request	Flask route
Open dashboard	GET /api/dashboard	dashboard()
Browse restaurant menu	GET /api/restaurants/{id}/menu	menu(restaurant_id)
Place cart order	POST /api/orders with JSON body	create_order()
Open reports	GET /api/reports	reports()
Admin adds menu item	POST /api/admin/restaurants/{id}/menu	add_menu_item() Blueprint route

A JSON body is used for changes. For example, checkout sends customer_id, payment_method, delivery_address, and an items array containing menu_item_id and quantity. Flask reads it with request.get_json().

4. MySQL to Frontend: Reading Data

A typical read route creates a dictionary cursor, executes a SELECT query, converts Decimal and DATETIME values into JSON-safe values, and returns jsonify(...). The browser receives an array or object and places it into the appropriate table, menu, or statistic area.

Restaurant browser example

1. JavaScript calls GET /api/restaurants.
2. Flask joins RESTAURANT with MENU_ITEM to count available dishes.
3. MySQL returns restaurant name, address, rating, and item count.
4. Flask returns JSON.
5. JavaScript creates a restaurant card for every returned record.

Dashboard example

The dashboard route reads the total customer count, total order count, total revenue, and active delivery count. It also reads recent orders from **vw_delivery_status_report**. That MySQL view already joins customer, restaurant, order, and delivery information, so Flask receives a report-ready result set.

Why JSON conversion matters

MySQL DECIMAL values arrive in Python as Decimal objects and DATETIME values arrive as datetime objects. json_value() changes Decimal to float and datetime to an ISO-style text value before jsonify() creates the response.

5. Order Placement: Frontend to MySQL

The checkout route is the most important write operation. It makes sure an order is complete and valid before it commits anything to the database.

Validate request	Check customer and menu	Start transaction	Insert linked records	Commit or rollback

Step	What Flask does	Why it matters
Validate	Checks customer id, positive quantities, payment method, delivery-address length, duplicate cart items, and single-restaurant rule.	Stops malformed requests before they reach MySQL.
Verify	Reads customer and MENU_ITEM records. It checks availability and locks selected menu rows for the transaction.	Prevents an unavailable item from being ordered.
Insert	Creates ORDERS, then ORDER_ITEM rows. It reads the trigger-calculated total and inserts PAYMENT and DELIVERY.	Every child record uses the same order_id.
Finish	Calls commit() after all writes succeed. Any exception calls rollback().	No partial order remains after a failed checkout.

6. What MySQL Does Independently

Flask does not replace database logic. It works with the database features already defined in the DBMS project. This is important during a viva because responsibilities stay clear: Flask handles requests and presentation; MySQL protects and organizes data.

MySQL feature	Effect when Flask uses the database
Foreign keys	Prevent records from pointing to a customer, restaurant, order, or menu item that does not exist.
UNIQUE and CHECK constraints	Reject duplicate customer email or phone values and invalid numeric values.
trg_orderitem_before_insert	Rejects an order item when the menu item is unavailable.
trg_orderitem_after_insert/delete	Recalculates ORDERS.total_amount after order items change.
trg_delivery_after_update	Updates the parent order to DELIVERED when a delivery becomes DELIVERED.
Views	Give Flask prebuilt delivery and restaurant-sales report data.

The documented sp_place_order procedure can handle one item. FoodFlow uses its own safe transaction by default because the cart supports multiple items. USE_STORED_PROCEDURE=true can enable the procedure only when the local procedure exactly matches the documented definition.

7. Login, Roles, and Admin Blueprint

The database schema does not include an application USER table, so the web app adds a lightweight administrator login using Flask sessions. APP_ADMIN_USERNAME and APP_ADMIN_PASSWORD stay in .env. On a successful login, Flask stores role=admin in the signed session cookie.

Route protection

api_route() checks whether the session has a role. Unauthenticated API calls return HTTP 401. The administrator Blueprint also calls require_admin(), which returns HTTP 403 for non-admin users. The login and logout endpoints remain available so a user can create or end a session.

Endpoint group	Access
/api/auth/login, /api/auth/logout, /api/auth/me	Authentication functions
/api/dashboard, /api/restaurants, /api/orders	Signed-in session required
/api/reports and /api/admin/...	Administrator session required

This application login complements DCL. MySQL DCL controls database accounts and permissions; the Flask login controls who can use the website interface.

8. Validation, Security, and Error Handling

Flask validates every value received from JavaScript before using it. The backend never trusts values simply because they came from the project interface.

Protection	Implementation
SQL injection protection	Every MySQL query uses %s placeholders with a separate parameters tuple.
Input validation	positive_int(), enum checks, address-length checks, email check, price and rating checks.
Credential handling	.env is ignored by Git. .env.example contains placeholders only.
Session safety	Secret key signs the session. HttpOnly and SameSite settings reduce cookie exposure.
Database failures	api_route catches mysql.connector Error and returns a safe message instead of exposing SQL details.
Transaction safety	Checkout rolls back after an exception before returning an error response.

For a production system, passwords should be stored as password hashes in a user table, HTTPS should be enabled, and login attempts should be rate limited. The existing version is appropriate for a college demonstration because its configuration is private and all management actions require the administrator session.

9. How to Explain It During a Demo

Use this short sequence when presenting the project.

Demo step	What to say
1. Start Flask	Flask reads connection and login settings from .env, then serves the HTML interface.
2. Sign in	The login route creates an administrator session. Protected routes reject unauthenticated requests.
3. Browse a menu	JavaScript fetches restaurant and menu data. Flask executes parameterized SELECT queries and returns JSON.
4. Place an order	The browser sends cart data as JSON. Flask validates it, opens a transaction, inserts linked records, and commits only after success.
5. Check reports	Flask queries MySQL views. The frontend converts the JSON report into visible sales and delivery information.
6. Update delivery	Flask updates DELIVERY. The MySQL trigger updates ORDERS when delivery becomes DELIVERED.

Main files to show the evaluator: app.py, admin_routes.py, templates/index.html, static/js/app.js, .env.example, and README.md.

Conclusion

Flask is the bridge between the browser and the existing MySQL database. It receives requests, validates them, runs safe database operations, returns JSON, manages the login session, and leaves core integrity rules inside MySQL.